

Mathematical Image Processing

In-person assignment 02, 06.05.2025

Summer semester 2026

The programming sheets are in-person assignments during the exercise. Please bring your own computer.

If you have questions concerning the exercise please refer to Jonathan Laubmann via jona.laubmann@fau.de.

Topics of this week:

- Image scaling
- Fourier transform
- Filtering
- Convolution

— AUFGABE 1: Image upscaling

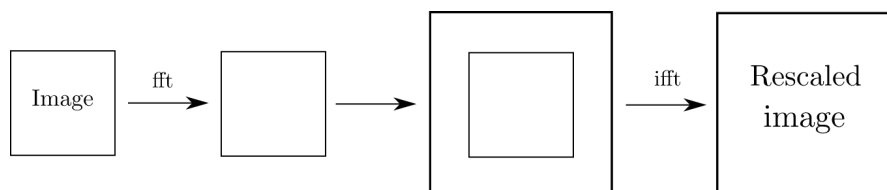
The goal of this exercise is to manipulate image grids and pixel coordinates by upscaling a given image.

1. Rescaling by duplication

The most naive method consists in copying the pixels of an image to make it bigger. Write a function `I_resize = resize_copy(I,factor)` that rescales image `I` by an integer factor `factor` by copying the pixel values accordingly.

2. Rescaling by zero padding

It is also possible to interpolate grids in the frequency domain (i.e., the Fourier space) with a method known as *zero-padding*. It consists in upscaling the Fourier transform of the image by adding extra zeros to the high frequencies.



Write a function `I_zeropad = zero_padding(I,factor)` that implements the zero-padding method.

Note:

- The Fourier transform for images is performed using the `fft2` function, the inverse Fourier transform with `ifft2` from `scipy.fft`.

- Please note that Scipys' Fourier transform puts the high frequencies at the middle, so if you want the low frequencies to be centered use the `fftshift` function.
- The Fourier transform is complex, so remember to use the real part to get a real image at the end.
- To keep the gray levels in the same range through the transform, it is better to multiply the transformed image by `factor`².

3. Rescaling with interpolation filters

Rather than copying the pixels values as in the `resize_copy` function, it is more sophisticated to interpolate the pixel values to have a smoother image via *spatial convolution* with (for example) the following filters:

$$\textbf{tent:} \quad h(x) = \begin{cases} 1 - |x| & \text{if } |x| \leq 1, \\ 0 & \text{else.} \end{cases} \quad (1)$$

$$\textbf{bell:} \quad h(x) = \begin{cases} -x^2 + \frac{3}{4} & \text{if } |x| \leq \frac{1}{2}, \\ \frac{1}{2} \left(|x| - \frac{3}{2}\right)^2 & \text{if } \frac{1}{2} < |x| < \frac{3}{2}, \\ 0 & \text{else.} \end{cases} \quad (2)$$

$$\textbf{Mitchell-Netravali:} \quad h(x) = \begin{cases} \frac{7}{6}|x|^3 - 2|x|^2 + \frac{8}{9} & \text{if } |x| < 1, \\ -\frac{7}{18}|x|^3 + 2|x|^2 - \frac{10}{3}x + \frac{16}{9} & \text{if } 1 \leq |x| < 2, \\ 0 & \text{else.} \end{cases} \quad (3)$$

Write a function `I_filter = resize_filter(I, factor, filter)` that performs the upscaling by interpolation with the given filter.

Note:

- For each pixel you have to perform the spatial convolution with the given filter in both directions, i.e. on the row and the columns of the image to be upscaled.
- Mind the image borders when performing the convolution!

— AUFGABE 2: Image filtering —

The goal of this exercise is to manipulate basic image filtering using spatial convolution filters and the median filter.

1. Write a function `I_filter = image_filter(I, filter)` that performs the spatial convolution of a (possibly noisy) image `I` with one of the following convolution
-

filters:

$$m = \frac{1}{25} \begin{pmatrix} 1 & 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 & 1 \end{pmatrix} \quad g = \frac{1}{81} \begin{pmatrix} 1 & 2 & 3 & 2 & 1 \\ 2 & 4 & 6 & 4 & 2 \\ 3 & 6 & 9 & 6 & 3 \\ 2 & 4 & 6 & 4 & 2 \\ 1 & 2 & 3 & 2 & 1 \end{pmatrix}$$

$$\nabla = \begin{pmatrix} 0 & -1 & 0 \\ -1 & 0 & 1 \\ 0 & 1 & 0 \end{pmatrix} \quad \Delta = \begin{pmatrix} 0 & 1 & 0 \\ 1 & -4 & 1 \\ 0 & 1 & 0 \end{pmatrix}$$

Here, m is the *averaging filter* and g is an approximation of the *Gaussian filter*, while ∇ and Δ are approximations of the amplitude of *gradient* and *Laplacian* of the image.

2. Write a function `I_median = median_filter(I,size)` that performs median filtering of the image `I` by taking for each pixel the median value over a surrounding square $N(i)$ of size `size` of pixel i as follows:

$$I_{\text{median}}(i) = \text{median}\{I_k \in N(i)\}$$

Try different window sizes and observe the immediate effect.

Note: The median filter is especially well suited to denoise *salt and pepper* or *impulsive noise* that you can generate with the following function `add_sp_imnoise` for uint8 images.

```

import random
import math

def add_sp_innoise(img):
    # Getting the dimensions of the image
    row , col = img.shape

    # Randomly pick some pixels in the
    # image for coloring them white
    number_of_pixels = random.randint(math.floor(0.1*row*col), 0.4*row*col)
    for i in range(number_of_pixels):
        # Pick random coordinates
        y_coord=random.randint(0, row - 1)
        x_coord=random.randint(0, col - 1)

        # Color that pixel to white
        img[y_coord][x_coord] = 255

    # Randomly pick some pixels in
    # the image for coloring them black
    number_of_pixels = random.randint(math.floor(0.1*row*col), 0.4*row*col)
    for i in range(number_of_pixels):
        # Pick random coordinates
        y_coord=random.randint(0, row - 1)
        x_coord=random.randint(0, col - 1)

        # Color that pixel to black
        img[y_coord][x_coord] = 0

    return img

```